Lua VM 表写入指令解析（OP_SETI / OP_SETTABLE）

在 Lua VM 中，对表（table）的写入是最关键的操作之一。今天我们讲两条指令：

- OP_SETI：表写入（整数 key，常用于数组式访问）
- OP_SETTABLE：表写入（动态 key，最通用的写入方式）

这两条指令从表中取出目标位置，然后写入寄存器或常量池的值。区别主要在 **key 的来源和访问路径**。

1. OP_SETI：整数 key 写入

```
// A B C   R[A][B] := RK(C)
L_OP_SETI: {
    StkId ra = RA(i);      // 获取寄存器 A 中的 table
    int hres;
    int b = GETARG_B(i);  // key，整数
    TValue *rc = RKC(i);  // value
    luaV_fastseti(s2v(ra), b, rc, hres);
    if (hres == HOK)
        luaV_finishfastset(L, s2v(ra), rc);
    else {
        // 慢路径
        TValue key;
        setivalue(&key, b);
        Protect(luaV_finishset(L, s2v(ra), &key, rc, hres));
    }
    vmbreak;
}
```

1.1 RK 宏

```c
#define POS_k (POS_A + SIZE_A)
#define TESTARG_k(i) (cast_int(((i) & (1u << POS_k))))
#define RKC(i) ((TESTARG_k(i)) ? k + GETARG_C(i) : s2v(base + GETARG_C(i)))
```

- 判断 C 的来源：**寄存器** 或 **常量池**
- `RKC(i)` 返回实际的值指针

1.2 快速写入（fast path）

```c
#define luaV_fastseti(t, k, val, hres)
    if (!ttistable(t)) hres = HNOTATABLE;
    else luaH_fastseti(hvalue(t), k, val, hres);
#define luaH_fastseti(t, k, val, hres)
    Table *h = t;
    lua_Unsigned u = l_castS2U(k) - 1u;
    if ((u < h->asize)) {      // key 在数组范围
        lu_byte *tag = getArrTag(h, u);
        if (checknoTM(h->metatable, TM_NEWINDEX) || !tagisempty(*tag)) {
            fval2arr(h, u, tag, val); // 写入数组
            hres = HOK;
        } else
            hres = ~cast_int(u);       // 需要慢路径
    } else {
        hres = luaH_psetint(h, k, val); // hash 写入
    }
```

1.3 写屏障（GC 相关）

```c
#define luaV_finishfastset(L, t, v) luaC_barrierback(L, gcvalue(t), v)
```

- Lua GC 对象分三色：白 / 灰 / 黑
- **写屏障**确保增量 GC 正确追踪引用关系

2. OP_SETTABLE：动态 key 写入

```
// A B C   R[A][R[B]] := RK(C)
L_OP_SETTABLE: {
    StkId ra = RA(i);
    int hres;
    TValue *rb = vRB(i);  // key
    TValue *rc = RKC(i);  // value
    if (ttisinteger(rb)) {
        luaV_fastseti(s2v(ra), ivalue(rb), rc, hres); // key 是整数 → fast
path
    } else {
        luaV_fastset(s2v(ra), rb, rc, hres, luaH_pset); // key 类型不固定 → 普
通写入
    }
    if (hres == HOK)
        luaV_finishfastset(L, s2v(ra), rc);
    else
        Protect(luaV_finishset(L, s2v(ra), rb, rc, hres));
    vmbreak;
}
```

2.1 luaH_pset

```
int luaH_pset(Table *t, const TValue *key, TValue *val)
{
    switch (ttypetag(key)) {
    case LUA_VSHRSTR: return luaH_psetshortstr(t, tsvalue(key), val);
    case LUA_VNUMINT: return psetint(t, ivalue(key), val);
    case LUA_VNIL:    return HNOTFOUND;
    case LUA_VNUMFLT: {
        lua_Integer k;
        if (luaV_flttointeger(fltvalue(key), &k, F2Ieq))
            return psetint(t, k, val);
    } /* FALLTHROUGH */
    default:
        return finishnodeset(t, getgeneric(t, key, 0), val);
    }
}
```

- 根据 key 类型调用不同写入函数
- 关键区别：**OP_SETI 已知整数 key → 可直接 fast path**
- OP_SETTABLE 可能需要先计算 key → 可能走慢路径

指令	key 来源	快路径条件	慢路径
OP_SETI	指令编码整数	key 在数组范围	hash 或元方法
OP_SETTABLE	寄存器或常量池	key 是整数且在数组范围	各种 key 类型

本质区别：

- OP_SETI：key 已经是立即数 → 数组式快速访问
- OP_SETTABLE：key 运行期计算 → 需要类型判断 → 可能走慢路径